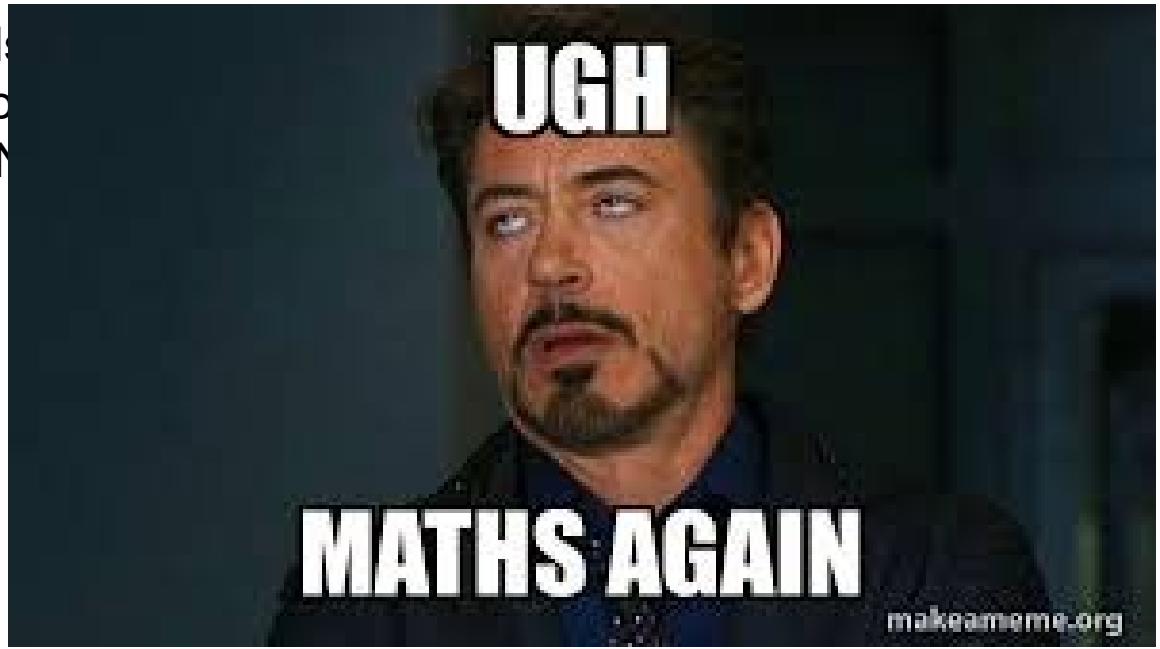
Week 11

Procedural Patterns

THE UNIVERSITY OF SYDNEY

This week

1) Fractals
2) Functio
3) Perlin N



This week (better)

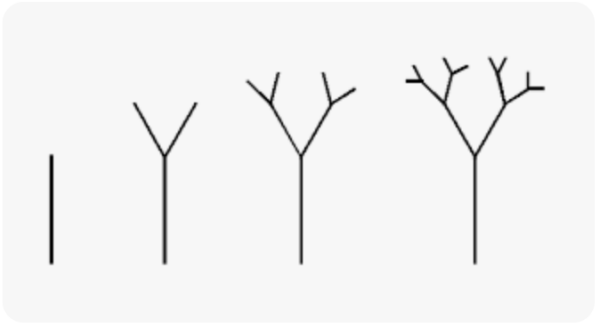
- Nature’s patterns (fractals)
- How to combine simple patterns to create complex ones (functions)
- A powerful technique to create smooth, natural randomness (Perlin noise)

~~Fractals and Self Similarity~~

Nature’s patterns

Fractals and Patterns

- Nature does not use harsh randomness.
- It has organised patterns that repeat at different scales.
- Think of a tree. Each branch is a “miniature” version of a tree.
- These self-repeating patterns are called *fractals.*



Fractals are everywhere!

- Real Life fractals!
    - Broccoli
    - Cauliflower
    - Leaves

Fractals in Design

- As Designers and Visual Artists, we want our digital work to look less “robotic” and more “natural”.

As Designers and Artists, how can we create and use these patterns?

~~Functions~~

Combining patterns to make better ones

Waves and Patterns

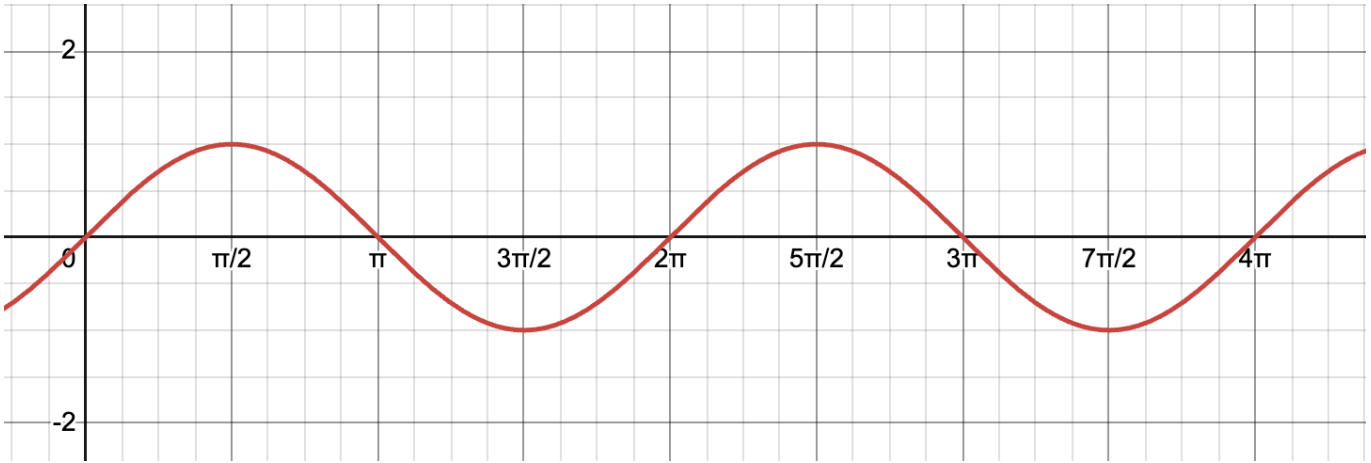
Now that you understand how fractals happen in real life, you understand the inspiration to many pieces of art that work with such patterns.

Lets explore how we can combine simple patterns to create more complex, visually compelling patterns.

Usually, we achieve this through layering. (Similar to how you would add different layers with different characteristics in photoshop)

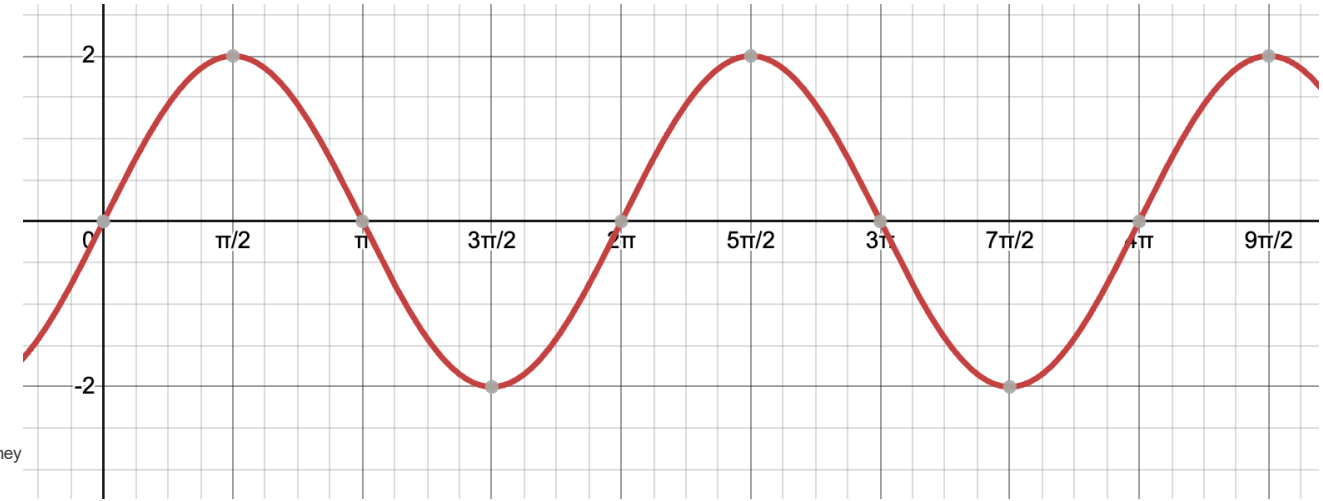
Waves

- Design and Maths overlap when you want to create more complex and interesting patterns.
- For someone with a maths background, it looks like a sine wave.
- This pattern is represented by $y = \sin x$



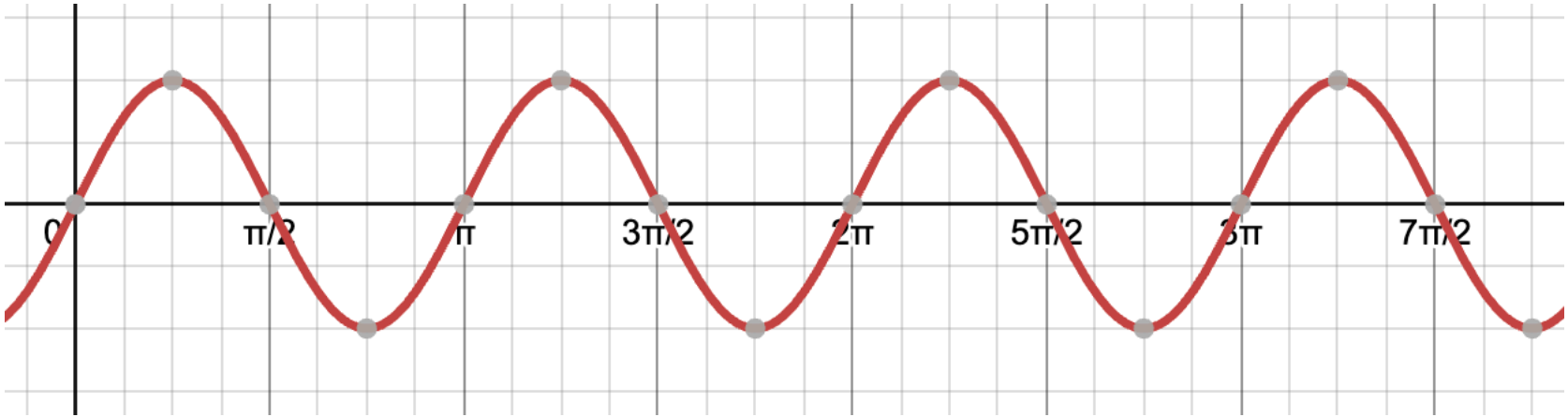
Waves: Amplitude

- The wave can be modified by adjusting their amplitude.
- The amplitude is the height of the weight, or how “dramatic” the pattern is.
- When you increase the amplitude (*Y = 2sin x)* the wave becomes more pronounced.



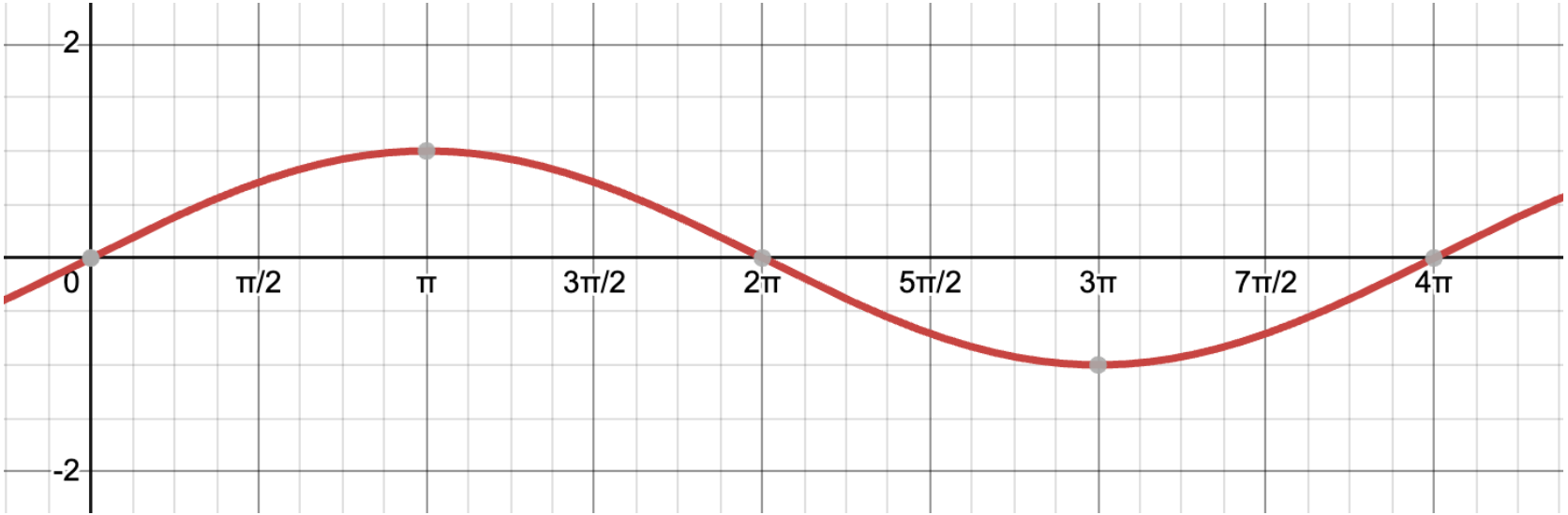
Wave: High Frequency

- You can also change how often you want the pattern to repeat.
- This is called frequency.
- High frequency, more repetitions ($y = sin(2x)$)



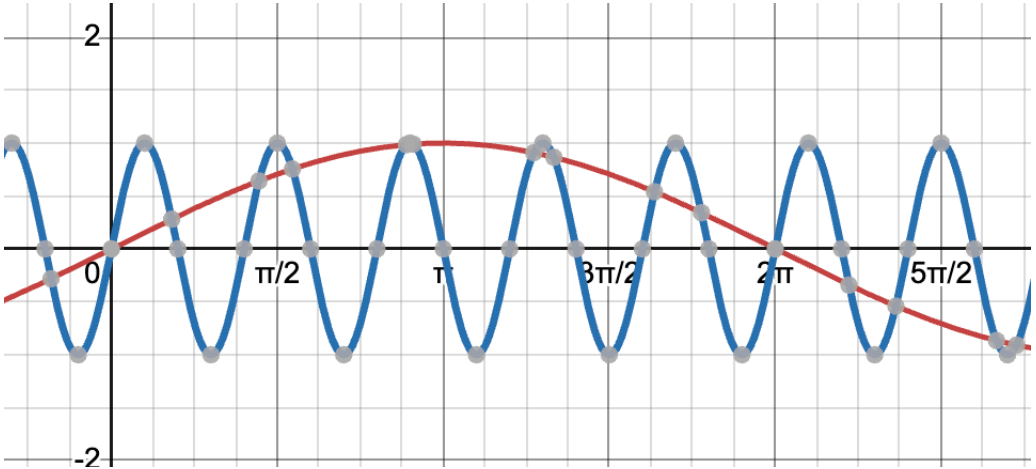
Wave: Low Frequency

- Low frequency can make a more stretched pattern.



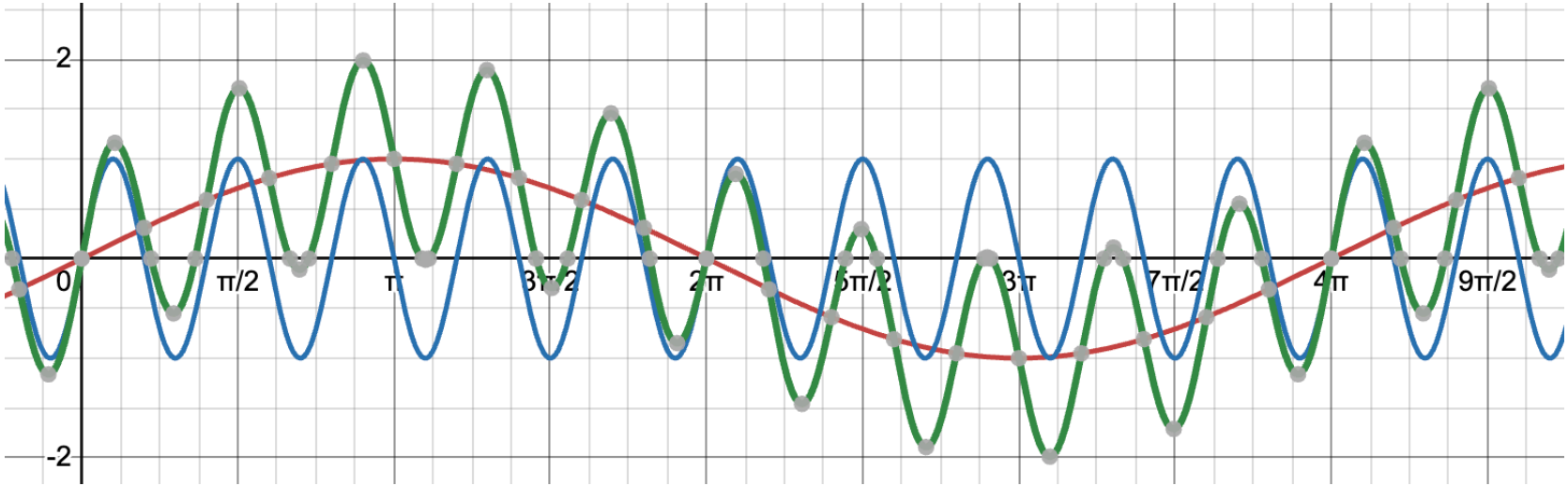
Waves and Layering

- Now, here is where we stop looking at robotic, repetitive patterns and bring art into place.
- When you have two different patterns and put them into the same picture, it looks like this:



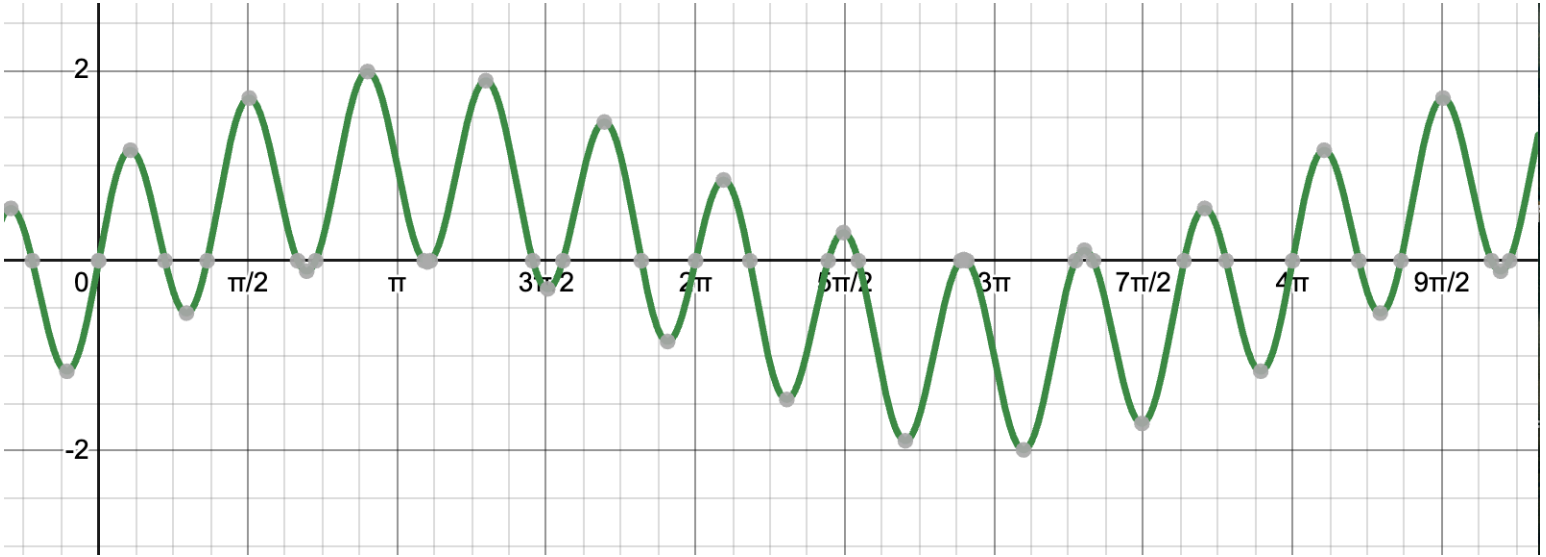
Waves and Layering

- Each wave represents a layer, and putting these two together, the result is a complex pattern that contains elements of both.



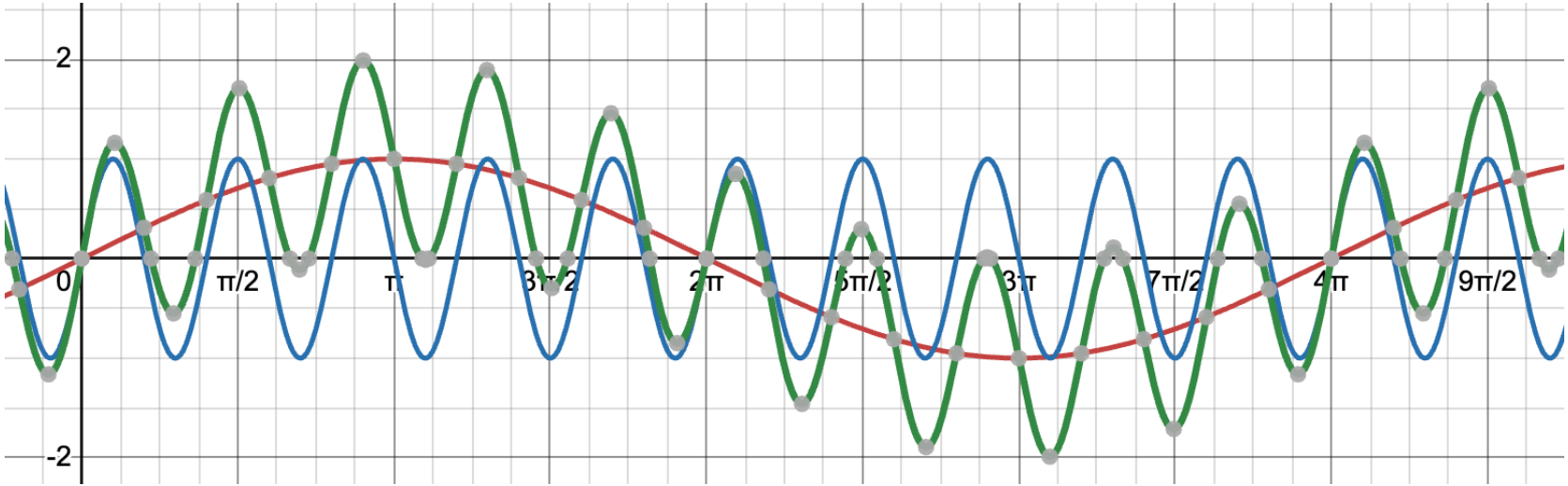
Waves and Layering

- Putting these patterns together, or putting one layer on top of the other (or adding the signals to those interested in the maths of all) produces a final pattern.



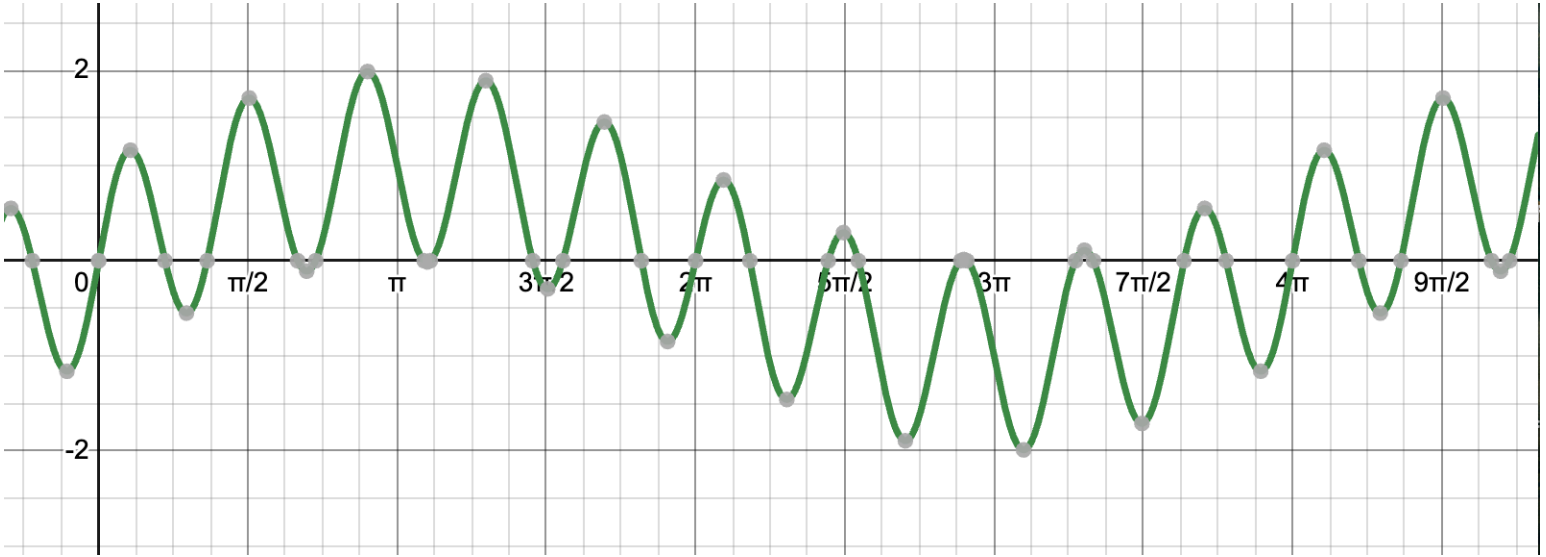
Waves and Layering

- You see how the wave follows the patterns of both signals we showed at the start?



Waves and Natural Patterns

- This is precisely how you create a “natural” pattern, by combining two artificial patterns to make them look like a natural pattern.



Closing Thoughts

- As a visual designer, you will see this layering concept often and need to get used to it.
- But the signals we just created are still… too perfect and regular.
- Nature has a structure but also variation and randomness.
- This is why we will learn about Perlin noise.

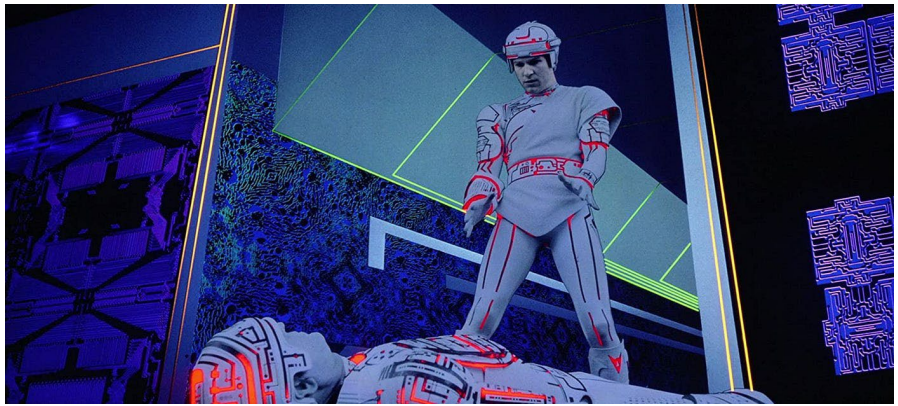
Perlin Noise: Creating Natural Randomness

Perlin Noise: Creating Natural Variations

- So you now know how maths can create patterns and randomness.
- But nature isn’t perfectly mathematical.
- It has structure and variation, but also it has some level of randomness.
- That is how Perlin noise helps. We will learn how it revolutionised Arts and Design.

Perlin Noise: Technical Concept

- Perlin noise is a way of generating "smooth" randomness.
- Ken Perlin introduced it in the 1980s to produce realistic-looking textures for computer-generated movie graphics.
- He won an Academy Award for developing and using Perlin noise in Computer Graphics in the movie Tron.



The Problem: Mechanical vs Natural Randomness

- To understand Perlin noise, let's compare how randomness is created.
- Imagine you have a JavaScript array of values.

```
let array = [1, 2, 3, 4, 5]
```

The Problem: Mechanical vs Natural Randomness

- To understand Perlin noise, let's compare how randomness is created.
- Imagine you have a JavaScript array of values.

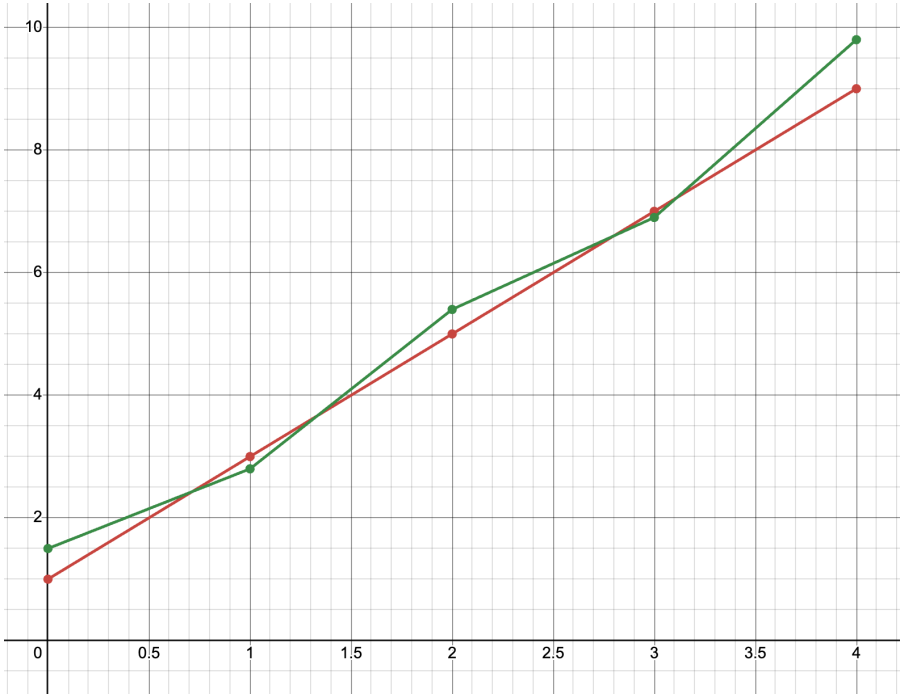
```
let array = [1, 2, 3, 4, 5]
```

- If we add random variations to these values (i.e. the *random ( )*), you get something like this.

```
let randomArray = [1.1, 1.8, 2.7, 3.5, 4.8]
```

Mechanical Randomness

- When we visualise those values, we have this:



What makes these random values mechanic is that there is no relationship between each value. They look truly random.

Mechanical Randomness: Better Example

- Lets see how a mechanical randomness can be created in p5.

```
let randomNumberArray = [];

function setup() {
  createCanvas(400, 400);

  for (let i = 0; i < 50; i++) {
    randomNumberArray.push(random(0, 100));
  }

  print(randomNumberArray);
}

function draw() {
  background(255);

  for (let i = 0; i < 50; i++) {
    line(i*4+100,300,i*4+100,randomNumberArray[i]+100);
  }
}
```

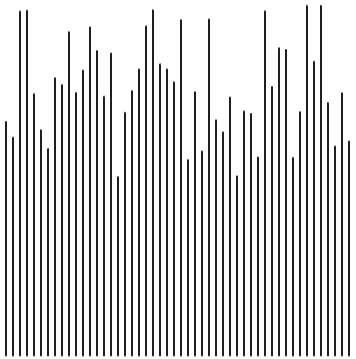
We will not look at this now, check it out in your own time.

Mechanical Randomness: Better Example

- Lets see how a mechanical randomness can be created in p5.

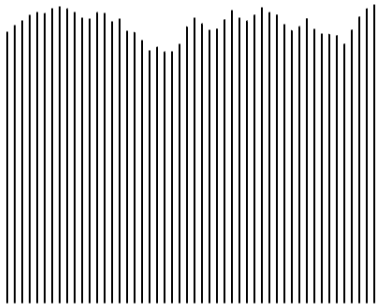
```
let randomNumberArray = [];

function setup() {
  createCanvas(400, 400);

  for (let i = 0; i < 50; i++) {
    randomNumberArray.push(random(0, 100));
  }

  print(randomNumberArray);
}

function draw() {
  background(255);

  for (let i = 0; i < 50; i++) {
    line(i*4+100,300,i*4+100,randomNumberArray[i]+100);
  }
}
```

Can you see how “mechanical” this looks?

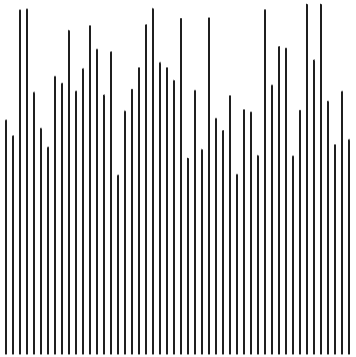


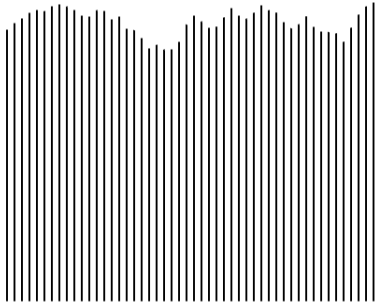
Natural Randomness: Example

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
    createCanvas(400, 400);

    for (let i = 0; i < 50; i++) {
        randomNumberArray.push(random(0, 100));
    }

    for (let i = 0; i < 50; i += perlinNoiseStep) {
        perlinNoiseArray.push(noise(i));
    }

    print(perlinNoiseArray);

}

function draw() {
    background(255);

    for (let i = 0; i < 50; i++) {
        line(i*4+100,300,i*4+100,perlinNoiseArray[i]*100+100);
    }
}
```



Better huh?





Natural Randomness: Example

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
    createCanvas(400, 400);

    for (let i = 0; i < 50; i++) {
        randomNumberArray.push(random(0, 100));
    }

    for (let i = 0; i < 50; i += perlinNoiseStep) {
        perlinNoiseArray.push(noise(i));
    }

    print(perlinNoiseArray);

}

function draw() {
    background(255);

    for (let i = 0; i < 50; i++) {
        line(i*4+100,300,i*4+100,perlinNoiseArray[i]*100+100);
    }
}
```

Note the use of *noise (i)*.

Natural Randomness: Example

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
    createCanvas(400, 400);

    for (let i = 0; i < 50; i++) {
        randomNumberArray.push(random(0, 100));
    }

    for (let i = 0; i < 50; i += perlinNoiseStep) {
        perlinNoiseArray.push(noise(i));
    }

    print(perlinNoiseArray);

}

function draw() {
    background(255);

    for (let i = 0; i < 50; i++) {
        line(i*4+100,300,i*4+100,perlinNoiseArray[i]*100+100);
    }
}
```

Note the use of *noise (i)*. What noise does is generate random numbers that are similar to each other to give a sense of smoothness.

Natural Randomness: Example

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
    createCanvas(400, 400);

    for (let i = 0; i < 50; i++) {
        randomNumberArray.push(random(0, 100));
    }

    for (let i = 0; i < 50; i += perlinNoiseStep) {
        perlinNoiseArray.push(noise(i));
    }

    print(perlinNoiseArray);

}

function draw() {
    background(255);

    for (let i = 0; i < 50; i++) {
        line(i*4+100,300,i*4+100,perlinNoiseArray[i]*100+100);
    }
}
```

Besides the use of noise, there are differences between these two loops, anyone can tell what is that?

Natural Randomness: Example

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
    createCanvas(400, 400);

    for (let i = 0; i < 50; i++) {
        randomNumberArray.push(random(0, 100));
    }

    for (let i = 0; i < 50; i += perlinNoiseStep) {
        perlinNoiseArray.push(noise(i));
    }

    print(perlinNoiseArray);

}

function draw() {
    background(255);

    for (let i = 0; i < 50; i++) {
        line(i*4+100,300,i*4+100,perlinNoiseArray[i]*100+100);
    }
}
```

Note how we increase the variable i by 0.1 and not by 1.

Natural Randomness: Example

```
let randomNumberArray = [];
let perlinNoiseArray = [];

let perlinNoiseStep = 0.1;

function setup() {
    createCanvas(400, 400);

    for (let i = 0; i < 50; i++) {
        randomNumberArray.push(random(0, 100));
    }

    for (let i = 0; i < 50; i += perlinNoiseStep) {
        perlinNoiseArray.push(noise(i));
    }

    print(perlinNoiseArray);

}

function draw() {
    background(255);

    for (let i = 0; i < 50; i++) {
        line(i*4+100,300,i*4+100,perlinNoiseArray[i]*100+100);
    }
}
```

Note how we increase the variable i by 0.1 and not by 1.

Noise needs a series of values that are close to each other to create smoothness.

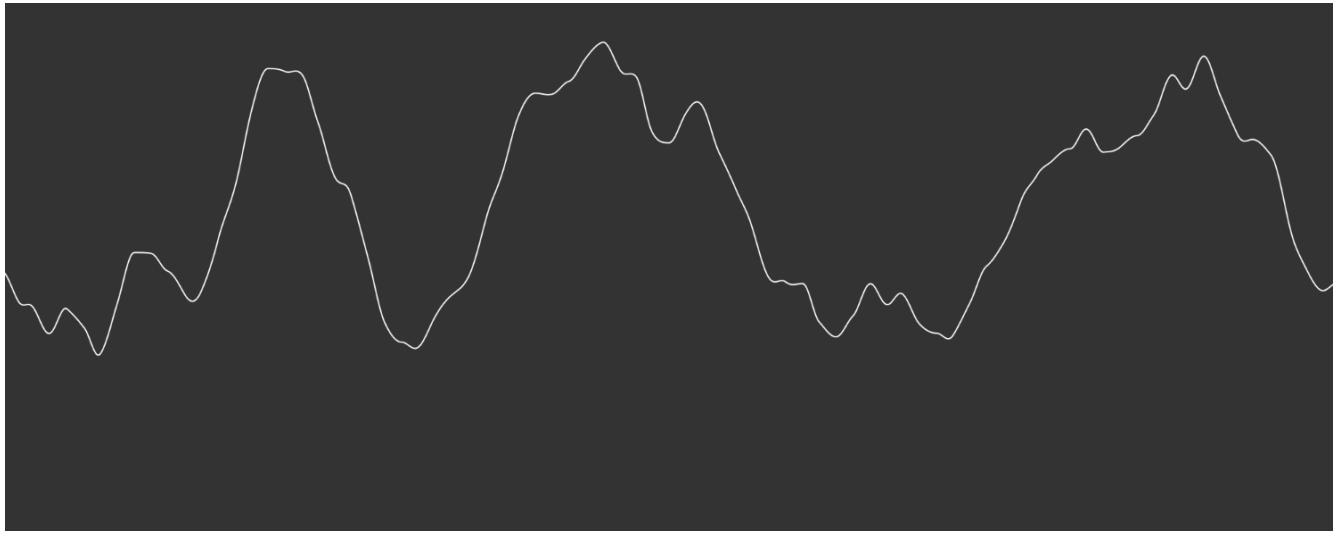
Think about it like this: between each abrupt change, you are filling in the blanks to make it look smooth.

More in the next slides.

Perlin Noise: in 1D and 2D

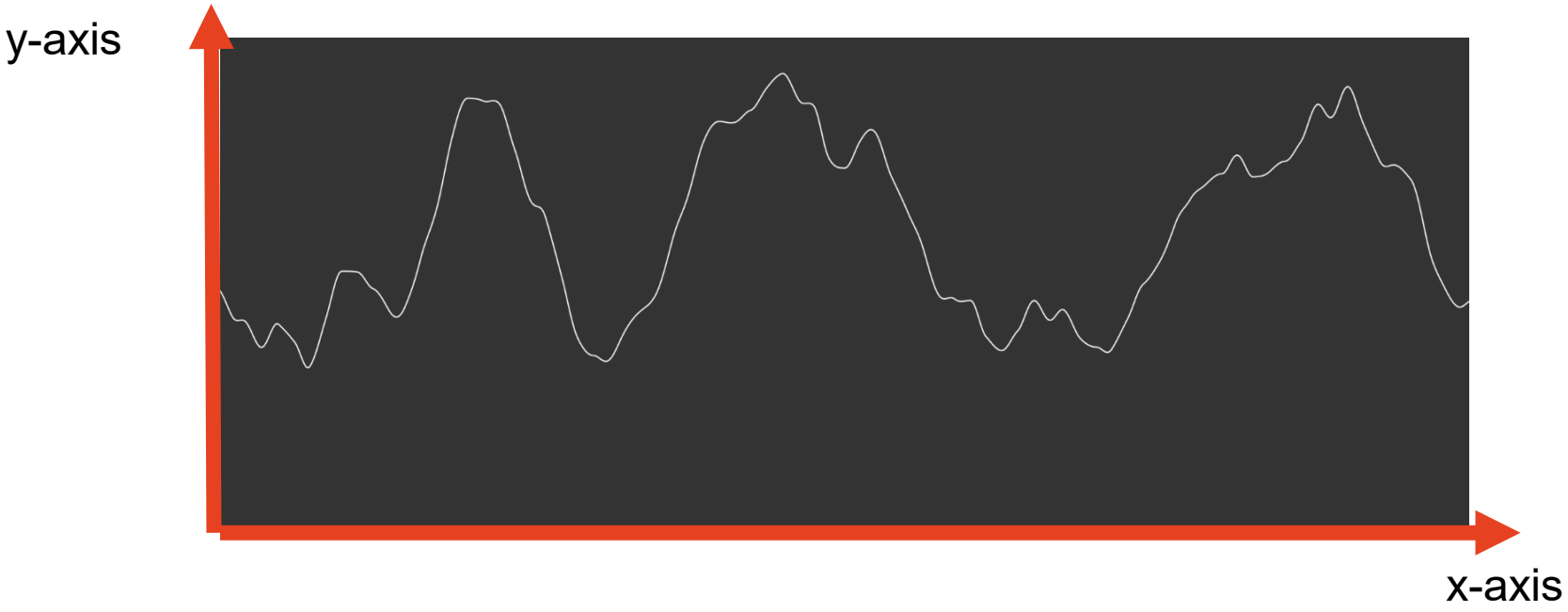
Perlin Noise in Detail: 1D

- The way Perlin noise works is as follows:
- Every time you run p5, it will create something like this:



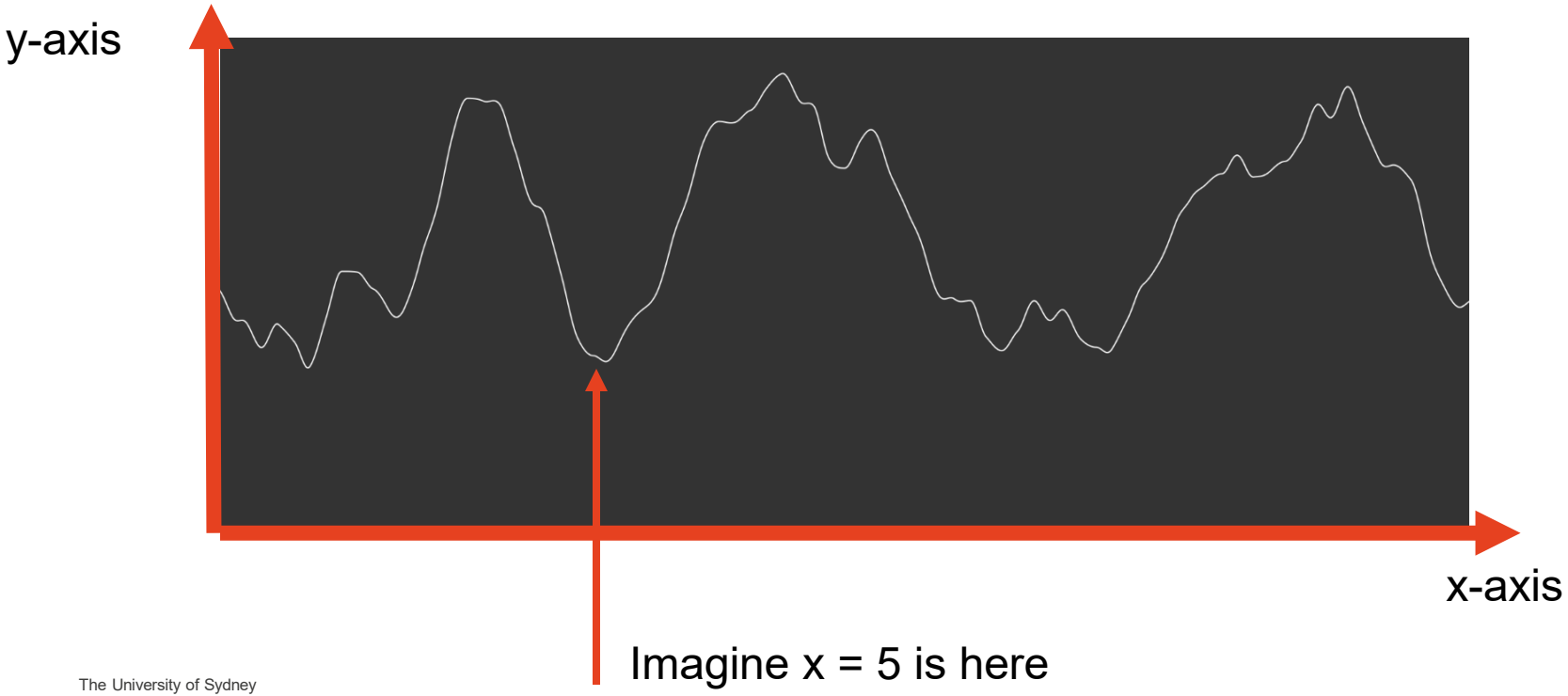
Perlin Noise in Detail: 1D

- Whenever you call the noise(value) function, it will search the value in the internal plot.



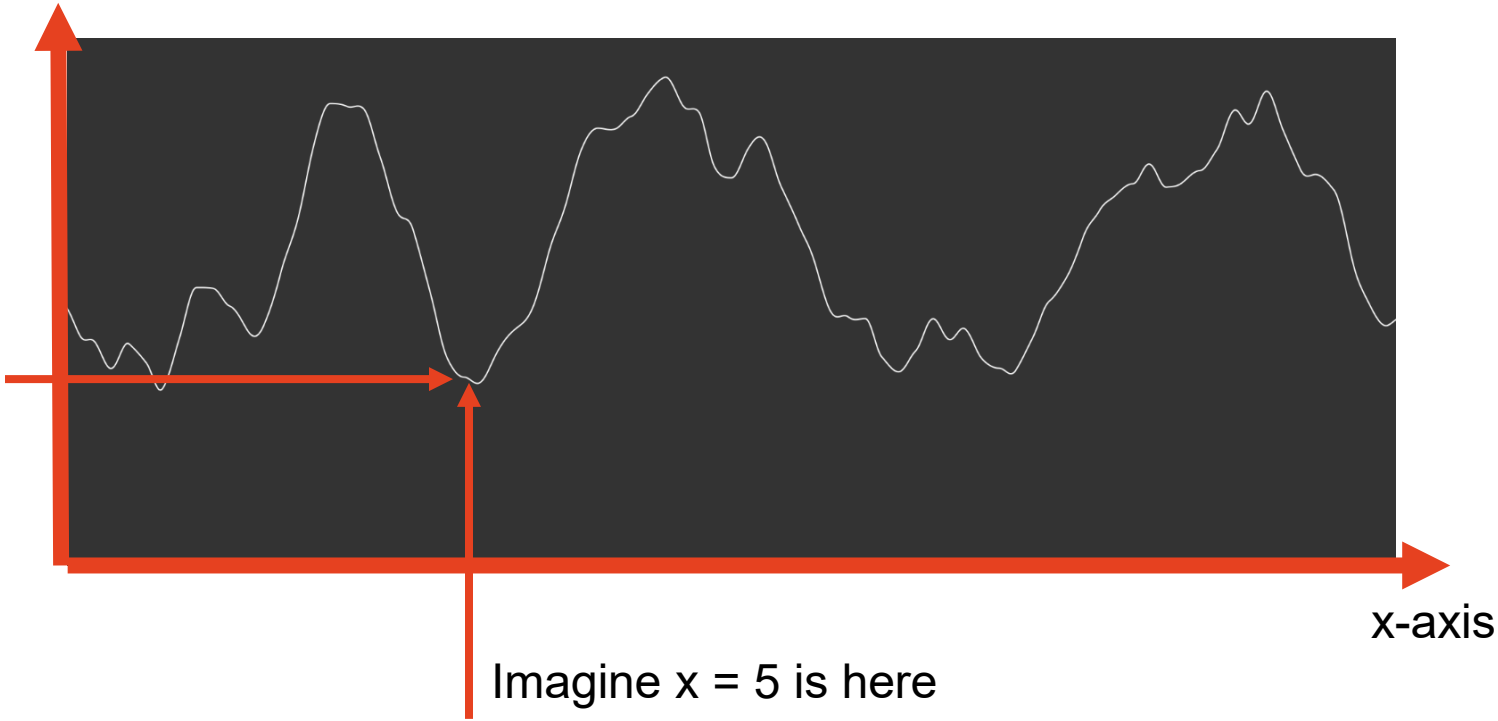
Perlin Noise in Detail: 1D

- Whenever you call the noise(value) function, it will search the value in the internal plot. Imagine value = 5.



Perlin Noise in Detail: 1D

- Whenever you call the noise(value) function, it will search the value in the internal plot. Imagine value = 5.



Perlin Noise 1D: Alternative view

- Another way of seeing Perlin noise is the following table.

Index	Noise Value
1	0.356
1.1	0.363
1.2	0.363
1.3	0.364
1.4	0.366
……	……
2	0.4

Perlin Noise 1D: Alternative view

- Note how the noise values have little to no difference between each index.

Index	Noise Value
1	0.356
1.1	0.363
1.2	0.363
1.3	0.364
1.4	0.366
……	……
2	0.4

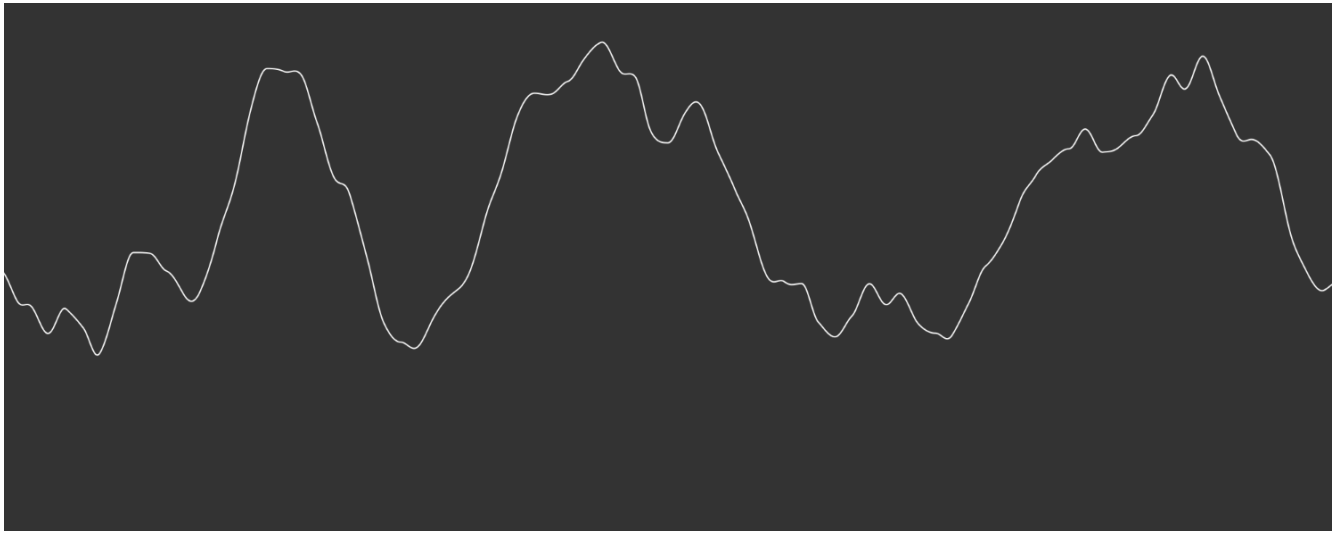
Perlin Noise 1D: Alternative view

- That is why we jump very little between values.
- Also note that the noise values are always below 1.
- Noise returns values between 0 and 1.
- That is why it gives such a smooth value versus random.

Index	Noise Value
1	0.356
1.1	0.363
1.2	0.363
1.3	0.364
1.4	0.366
……	……
2	0.4

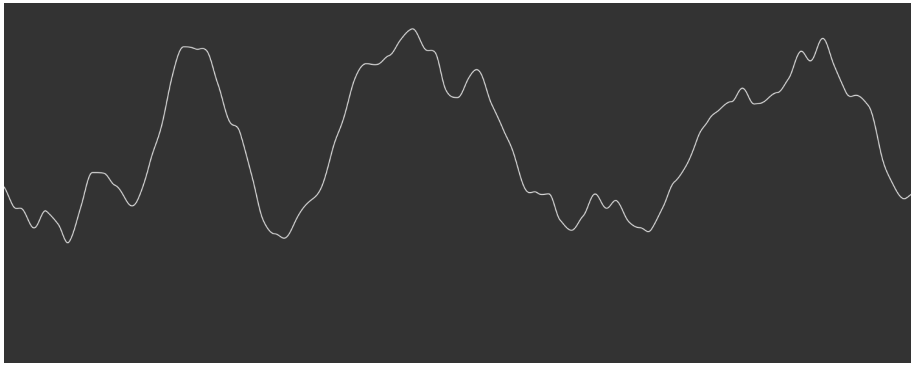
Question

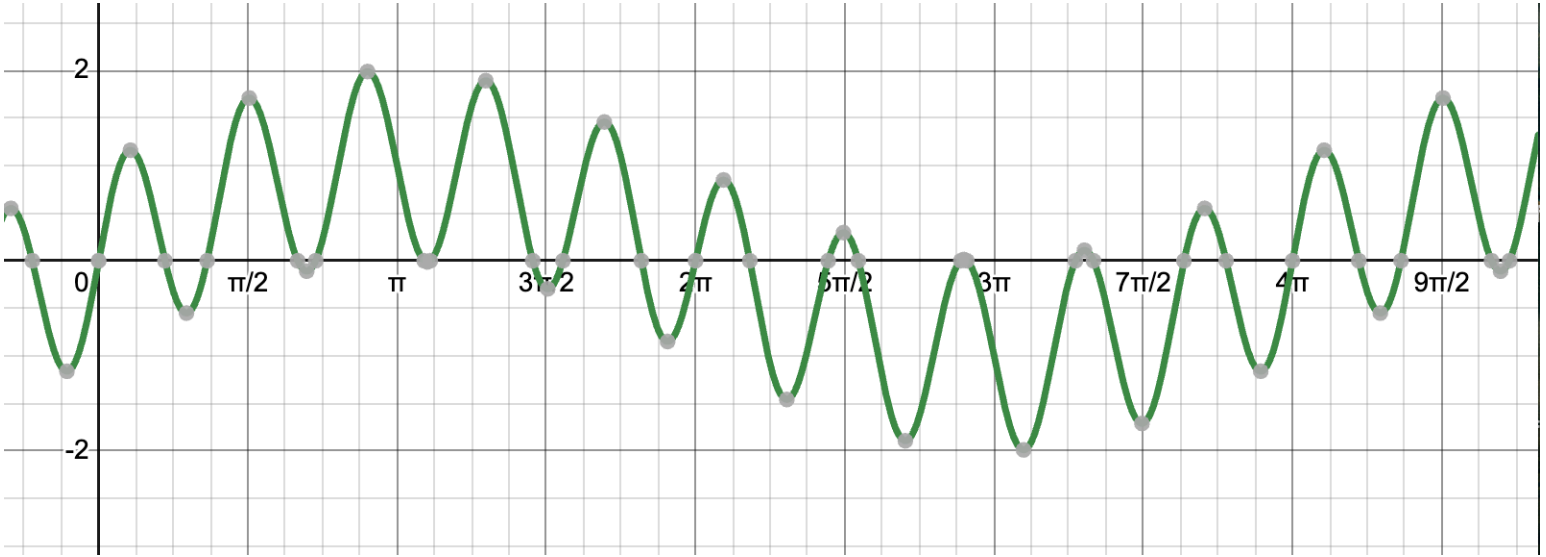
Did you notice anything about this line?



They are similar!

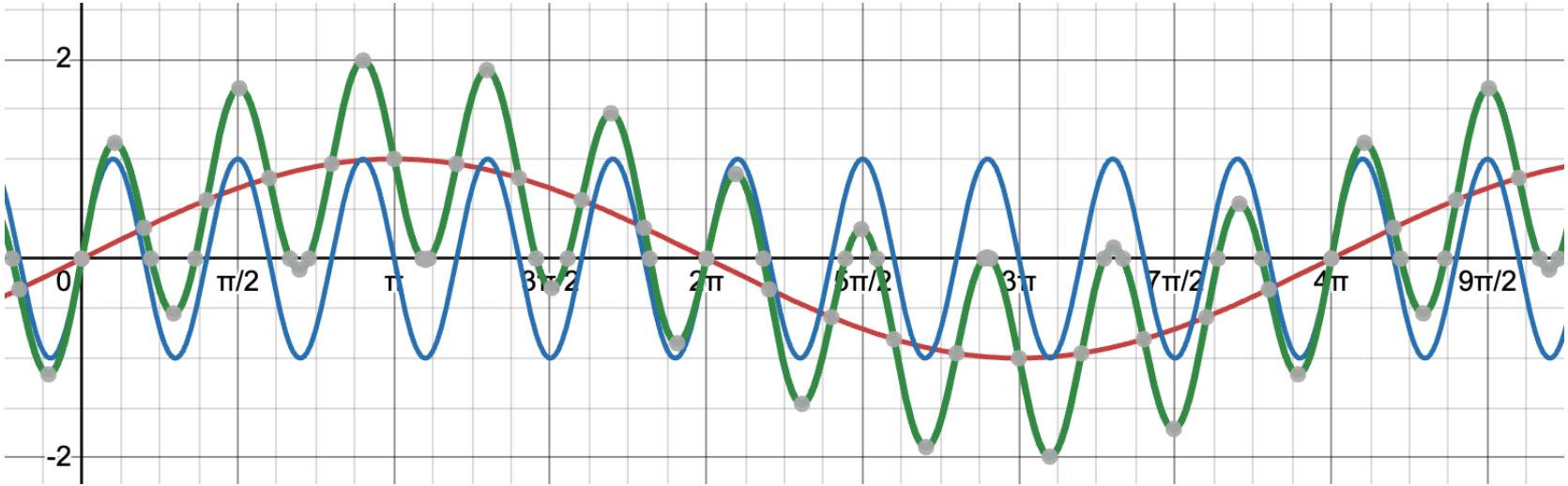
ish…





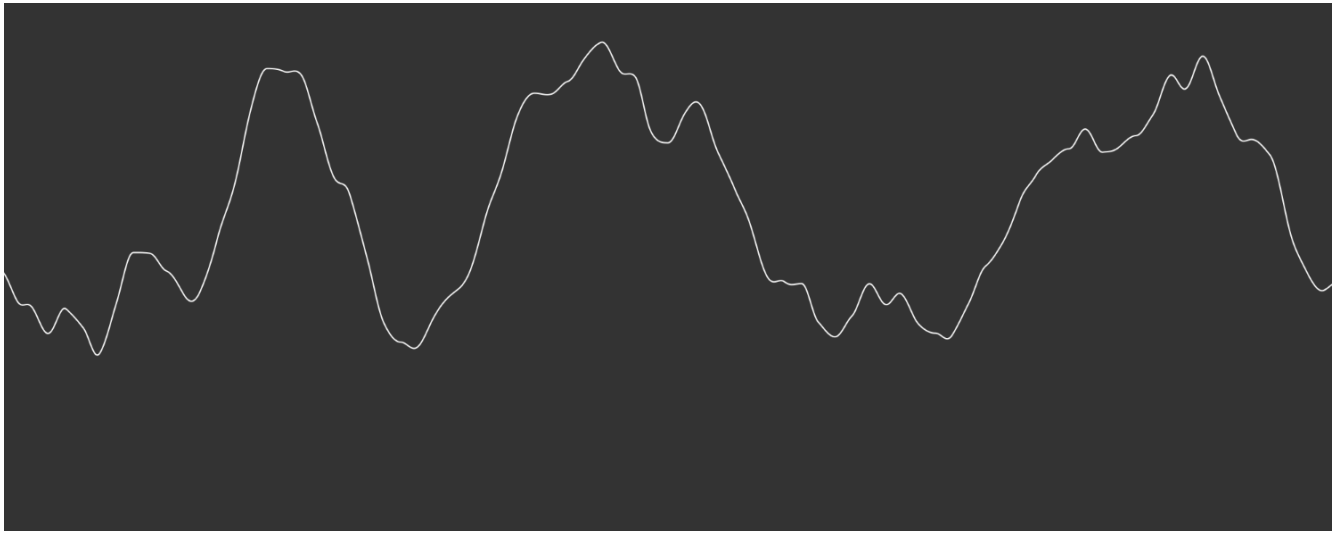
The Perlin Noise Algorithm First Creates various signals

- Through multiple maths it goes to the process of layering signals.



The Perlin Noise Algorithm First Creates various signals

- And adding randomness



Final Thoughts Perlin Noise 1D

- Of course, it is not just that.
- Multiple steps between the signal layering and randomness make up Perlin noise.
- Have a look at the Coding Train to understand more about what is behind it (and get ideas for your final project)
    - https://www.youtube.com/watch?v=Qf4dIN99e2w

What about Perlin Noise in 2D?

Yes there is Perlin Noise in 2D is a thing

Perlin Noise in Detail: 2D

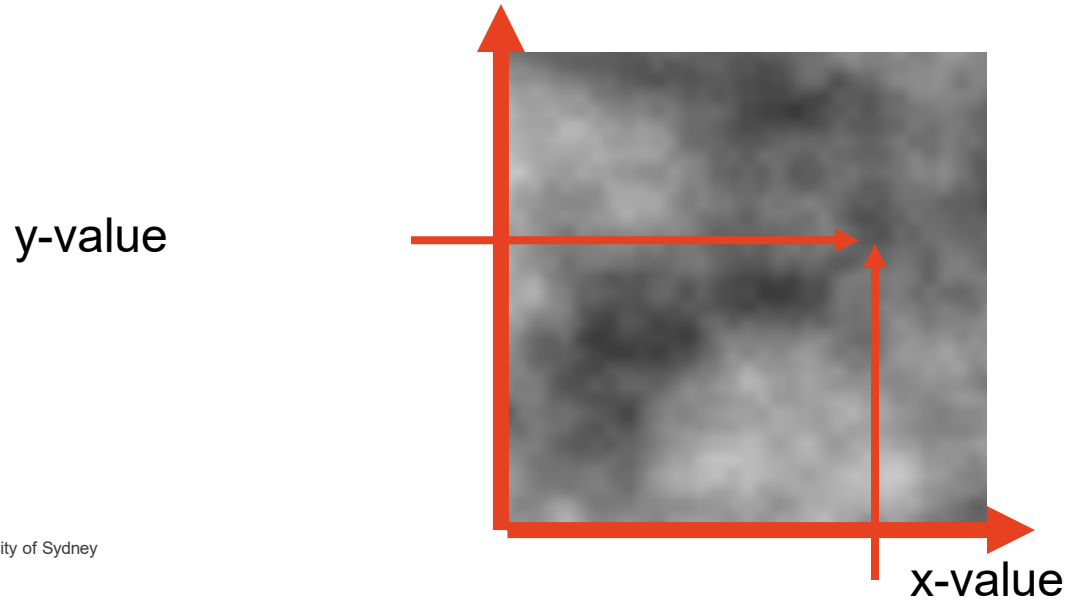
- There is similar functionality if you want a 2-dimensional value of Perlin Noise.
- When running your script, p5 will generate a random noise looking like this:

Perlin Noise in Detail: 2D

- You need to call *noise (xvalue, yvalue).*

Perlin Noise in Detail: 2D

- You need to call *noise (xvalue, yvalue).*

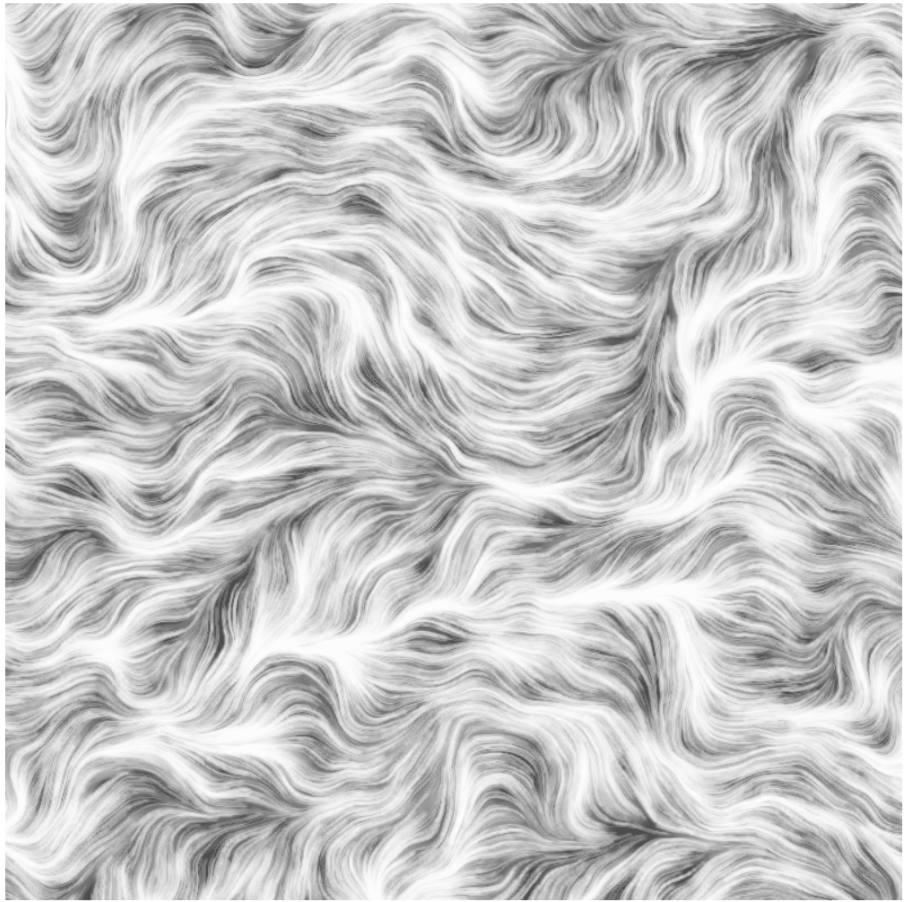


But how does this apply?

- Perlin noise is usually used to create artificial landscapes.

Art and Math

- A combination of 2D Perlin noise and math functions leads to fascinating art pieces.
- Try thinking about what type of pattern you want to develop, and if you spend some time on the internet, you will find a way to use Perlin noise to create the pattern!



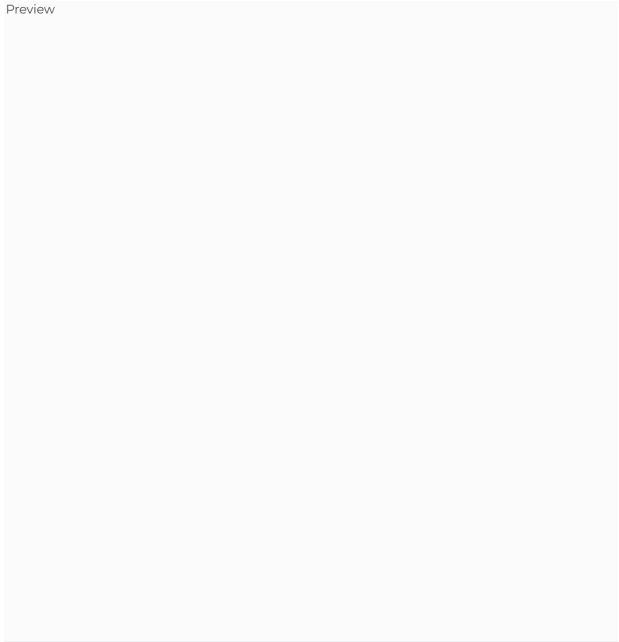
Closing Thoughts

- Remember, every time you run your sketch, p5 will generate **NEW** randomised plots, so no two sketches will look the same!
- Rather than teaching you how to create the pieces, we are teaching you the **tool** that will later help you create some pieces.
- It is up to **you** to search how can you use this tool for your creative adventures!

What about our final project?

Applications to the final project

- Here are some ideas of how you can use Perlin Noise for your final project:
    - Draw a tree and branch simulation using lines, and use Perlin noise to calculate the angles of each tree branch



Applications to the final project

- You can create a random walker

See you in the tutorial!!!!